

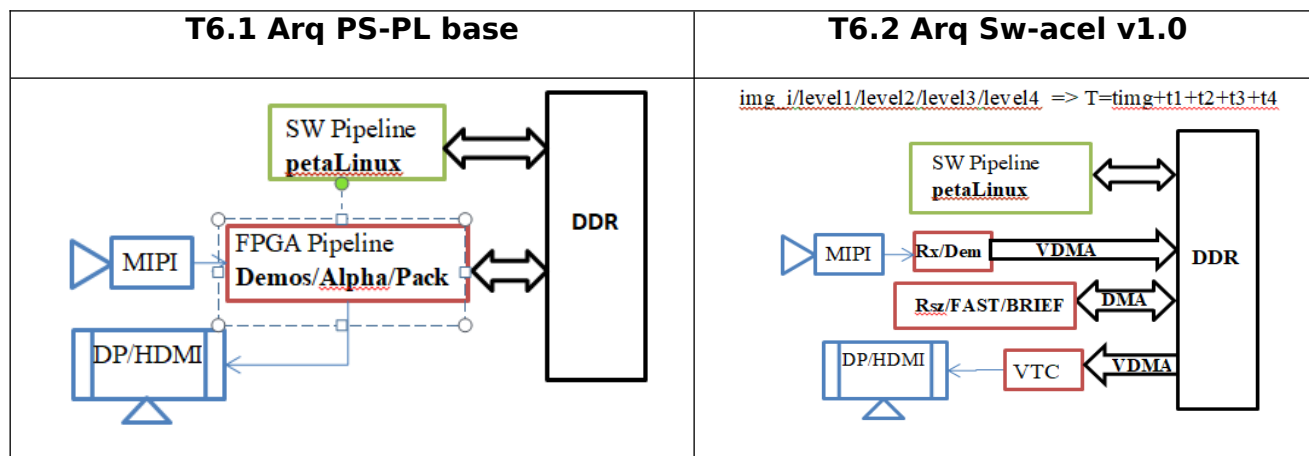
Integración arquitectura Sw-Acel v1.0

1. Objetivos

OBJ1. Identificación de los cuellos de botella en la integración del acelerador ORB con el pipeline Sw

OBJ2. Diseño del mecanismo de sincronización captura/PipelineSw

OBJ3. Generación de la arquitectura T6.2 Sw-acel v1.0

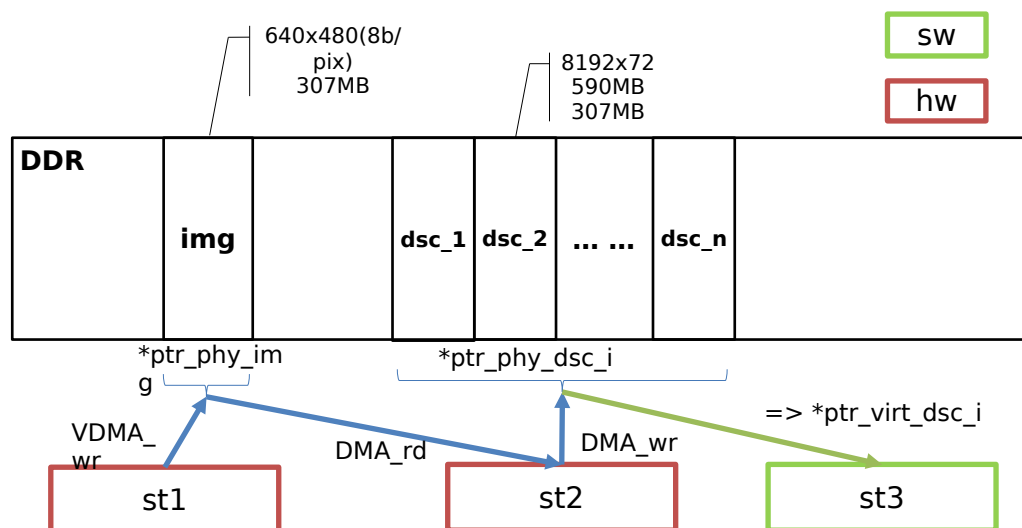
[illegible]

2. Integración ORBextractor (Acel.) en pipelineSw

Hay 3 etapas de procesamiento en la arquitectura T6.2

- St1: Captura de la imagen, acondicionamiento, conversión RGB2Y, envío a DDR via VDMA
 - o datos escritura: 640x480 (8b)
- St2: Extracción de descriptores en las n_escalas, supone N procesos de lectura/escritura via DMA de la imagen (lectura) y los descriptores (escritura).
 - o datos lectura: 640*480 (1B)
 - o datos escritura: $n_escalas * (8192 \text{ descrip } (64B) * 1 \text{ score } (4B) * 2 \text{ coord } (2B))$
8192 es el número máximo de descriptores que genera ORBextractor
- St3: Ejecución de thread de tracking (pipelineSW)
 - o datos lectura: $n_escalas * (8192 \text{ descrip } (64B) * 1 \text{ score } (4B) * 2 \text{ coord } (2B))$
 - o copia de los datos a una estructura específica

La transferencia de datos entre las distintas fases aparece en la siguiente figura:



- La transferencia de datos desde la st1 a la st2 se realiza intercambiando un puntero a la dirección física de la DDR ***ptr_phy_img**. Igualmente, los descriptores generados en la st2 se acceden mediante otro puntero a dirección física ***ptr_phy_dsc_i**. Estos punteros son manejados directamente por el Hw (VDMA y DMA).
- Para acceder a los datos desde el Sw (st3) es necesario conocer su dirección virtual ***ptr_virt_dsc_i**.

Ajustes Linux para compartir memoria física con el Hw

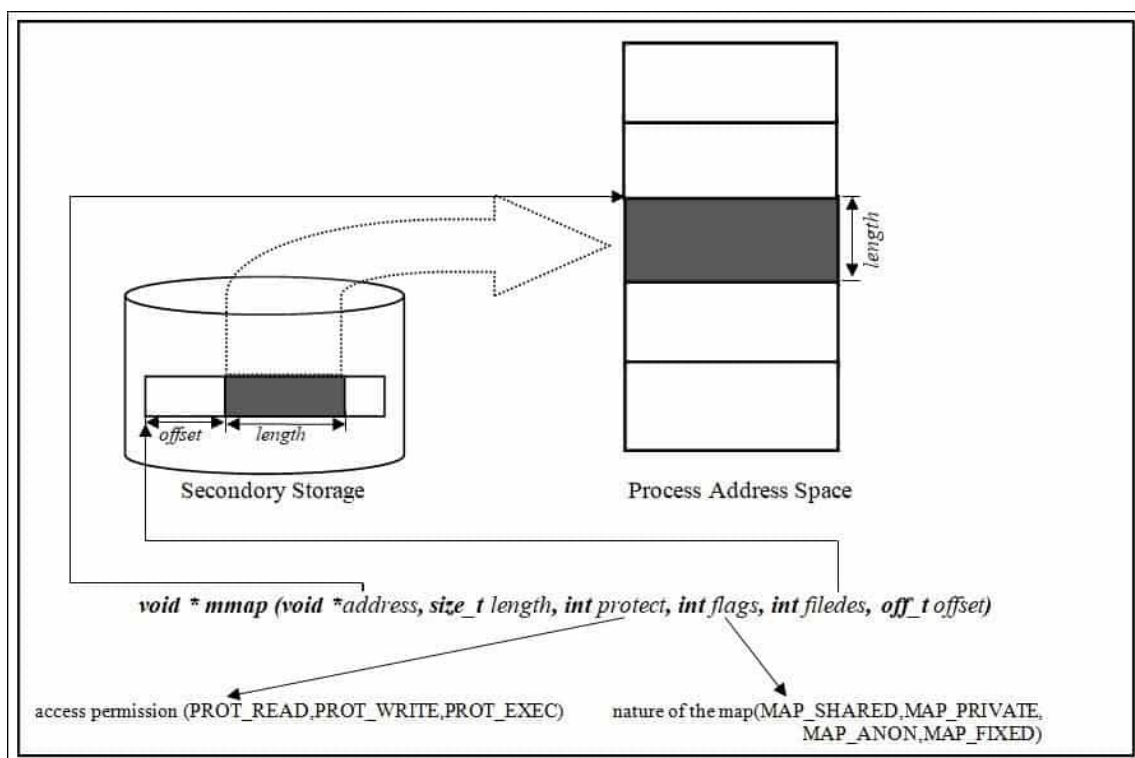
- (a) reservar una zona de memoria en el devicetree
- (b) utilizar alguna función del SO que nos devuelva la dirección virtual de una zona física de memoria (p.ej: **mmap**)

3. Análisis de alternativas

A) Utilización de mmap para acceder a la imagen escrita en DDR por el VDMA.

"mmap() creates a new mapping in the virtual address space of the calling process. The starting address for the new mapping is specified in *addr*. The *length* argument specifies the length of the mapping (which must be greater than 0)."

```
#include <sys/mman.h>
void *mmap(void addr[length], size_t length, int prot, int flags,
           int fd, off_t offset);
```



fuelle: https://linuxhint.com/using_mmap_function_linux/

- Si *addr*=0 /NULL el kernel elige la dirección (page-aligned) donde hacer el mapping
- The contents of a file mapping are initialized using *length* bytes starting at offset *offset* in the file (or other object)referred to by the file descriptor *fd*. *offset* must be a multiple of the page size as returned by **sysconf(_SC_PAGE_SIZE)** (4096 normalmente).
- The *length* argument specifies the length of the mapping (which must be greater than 0).

// SC: ejemplo, basado en
<https://github.com/hackndev/tools/blob/master/devmem2.c>
// fuente:
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842018/Linux+User+Mode+Pseudo+Driver>

```
#define GPIO_BASE_ADDRESS    0x81400000
#define GPIO_DATA_OFFSET    0
#define GPIO_DIRECTION_OFFSET    4
#define MAP_SIZE 4096UL
#define MAP_MASK (MAP_SIZE - 1)

int main() {
    int memfd; // SC: file descriptor
    void *mapped_base, *mapped_dev_base; // SC: *mapped_dev_base es la
    *virt_addr
    off_t dev_base = GPIO_BASE_ADDRESS;

    memfd = open("/dev/mem", O_RDWR | O_SYNC);
    // memfd = descriptor asociado a todo el mapa de memoria del Hw

    // 1) Map one page of memory into user space such that the device is
in that
    // page, but it may not be at the start of the page.
    // SC:
    // addr=0 el kernel elige la dirección (page-aligned) donde hacer el
map
    // offset = GPIO_BASE_ADDRESS & ~MAP_MASK (para que sea multiplo de
4096)
    //      = 0x8140_0000

    mapped_base = mmap(0, MAP_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
memfd, dev_base & ~MAP_MASK);

    // 2) get the address of the device in user space which will be an
offset
    // from the base that was mapped as memory is mapped at the start of a
page
    mapped_dev_base = mapped_base + (dev_base & MAP_MASK);

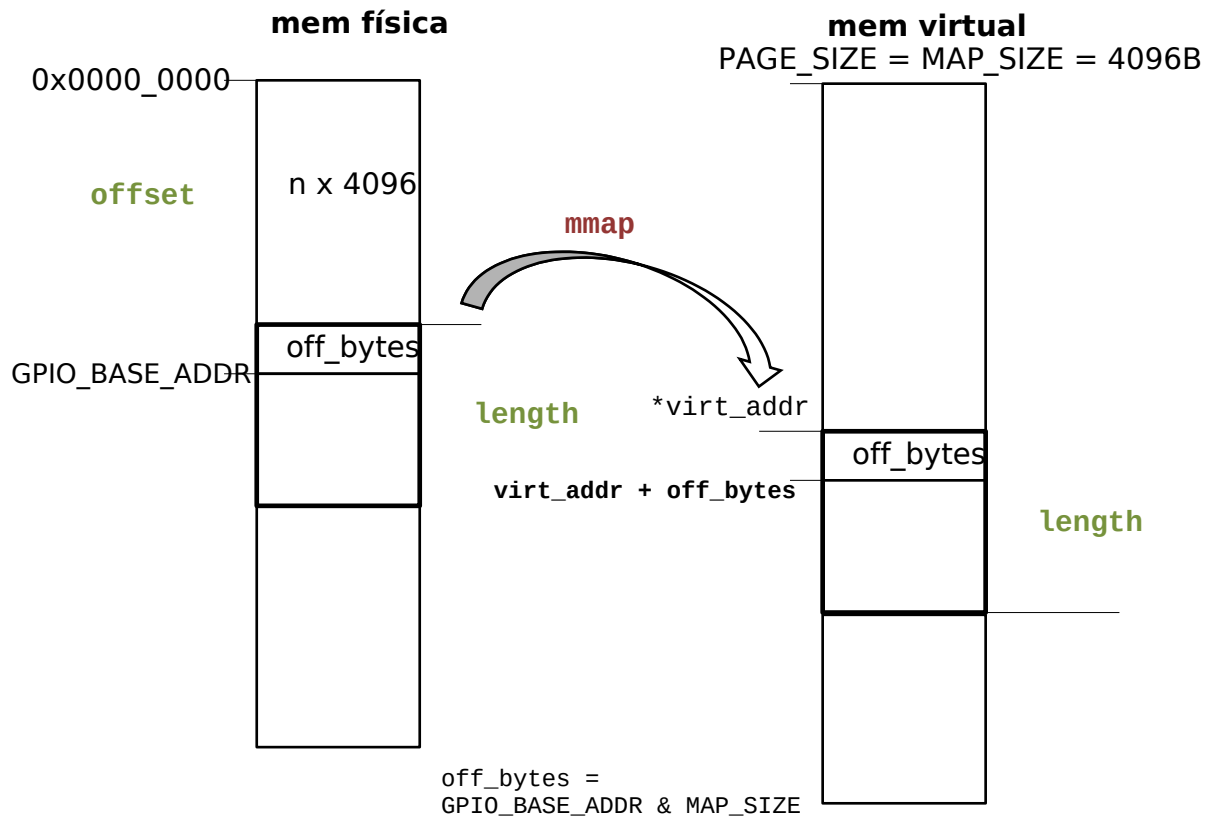
    // write to the direction register so all the GPIOs are on output

    *((volatile unsigned long *) (mapped_dev_base +
GPIO_DIRECTION_OFFSET)) = 0;

    // 3) toggle the output as fast as possible just to see how fast it
works
    while (1) {
        // If writes to multiple addresses were done:
        //     may need memory barriers i.e. need a driver
        //     caution with data being cached
        *((volatile unsigned long *) (mapped_dev_base + GPIO_DATA_OFFSET))
= 0;
        *((volatile unsigned long *) (mapped_dev_base + GPIO_DATA_OFFSET))
= 1;
    }

    // unmap the memory before exiting
    munmap(mapped_base, MAP_SIZE);

    close(memfd);
    return 0;
}
```



Problema: no se puede realizar un mapeo (mmap) del tamaño completo de la imagen/datos, sino que hay que ir mapeando por bloques de tamaño PAGE_SIZE (4096). Esto ralentiza el acceso a la imagen/datos.

código Víctor

[https://github.com/embention/MissionComputer/tree/feature/
%23121_SW_coprocessors_integration/items/zusp_15eg/items/sw_zusp_15eg/items/
sw_test_app_os/code](https://github.com/embention/MissionComputer/tree/feature/%23121_SW_coprocessors_integration/items/zusp_15eg/items/sw_zusp_15eg/items/sw_test_app_os/code)

configuración petalinux

[code](#) /[zusp_15eg_plnx](#) /[project-spec](#) /[meta-user](#) /[recipes-bsp](#) /[device-tree](#) /[files](#)
/system-user.dtsi

```
/include/ "system-conf.dtsi"
/ {
    reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;

        ranges;

        my_reserved_memory: my_reserved_region@600000000 {
            no-map;
            reg = <0x0 0x600000000 0x0 0x40000000>;
        };
    };
};
```

```
};
```

Aparentemente length no tiene restricciones más allá de que tiene que ser >0.

Cuando es dispositivo a mapear es la memoria: `memfd = open("/dev/mem", O_RDWR | O_SYNC);`

Es necesario mapear con tamaño máximo PAGE_SIZE. Si length es mayor aparece un error **BUS_ERROR Linux exception**

Posibles soluciones:

En este caso el **dispositivo es un uio** para mapear 256KB de la memoria OCM

```
1. char *uiod = "/dev/uio0";
2.
3. *fd = open(uiod, O_RDWR);
4. if (*fd < 1) {
5.     printf("Invalid UIO device file:%s.\n", uiod);
6.     return MAP_FAILED;
7. }
8.
9. // mmap the OCM memory into user space
10.     int pagesize= getpagesize();
11.     ocm_ptr = mmap(NULL, OCM_SIZE, PROT_READ|PROT_WRITE, MAP_SHARED, *fd,
        pagesize);
12.     if (ocm_ptr == MAP_FAILED) {
13.         printf("Mmap OCM failure.\n");
14.         return MAP_FAILED;
15.     }
```

1.a Post explicativo

fuelle: https://support.xilinx.com/s/question/0D52E00006hpMZASA2/how-to-change-memory-page-size-in-mmap-function?language=en_US

1b. Un warning sobre la utilización de uio

The advantage of using the uio driver is that you can use also the interrupts of the vdma. BUT there is a trap to it and it costed me two days to figure that out.

The uio framework can handle only one interrupt line per ip core. The vdma core has two (one for s2mm and one for mm2s) So in case you want to use interrupts your design wont work. You would need to build two dma cores into your design, one for s2mm one as mm2s, each with the other half deactivated and each bound to its own uio driver instance.

fuelle: https://support.xilinx.com/s/question/0D52E00007HjafISAR/write-and-read-to-vdma-channels-from-user-space-in-linux-application?language=en_US

1.c Otro post al respecto

https://support.xilinx.com/s/question/0D52E00006iHmHiSAK/petalinux20174-how-to-create-vdma-video-buffers?language=en_US

<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841683/Linux+Reserved+Memory>

B) Utilización del kernel driver udmabuf .

Es la opción utilizada